

# Smart Parking System - Full Project Overview & Guidelines

## 1. Project Overview

The Smart Parking System is a full-stack web application developed using React + TypeScript for the frontend, Spring Boot + MySQL for the backend, and JWT authentication for security. The system allows users to: reserve parking slots, make payments, monitor bookings, and manage parking availability in real time.

## 2. Main Technologies

Frontend: - React - TypeScript - Tailwind CSS - Axios - React Router - Lucide Icons Backend: - Spring Boot 3 - Spring Security - JWT Authentication - Spring Data JPA - Hibernate - MySQL - Swagger/OpenAPI - Lombok Additional: - FastAPI for AI recommendation service - Maven build system

## 3. Authentication Flow

1. User registers from frontend. 2. Spring Boot validates request. 3. Password is encrypted using BCrypt. 4. User data is stored in MySQL. 5. JWT token is generated. 6. Frontend stores token in localStorage. 7. Axios interceptor sends JWT automatically. 8. JwtAuthenticationFilter validates token for protected APIs.

## 4. User Features

- Register/Login - View parking locations - View available parking slots - Create booking - Cancel booking - Payment integration - Payment history - User dashboard - User profile

## 5. Admin Features

- Admin dashboard - Manage users - Manage parking slots - Change slot status - View all bookings - View all payments - Refund payments - Analytics dashboard - Management panel

## 6. Booking System Flow

1. User selects location. 2. User selects available slot. 3. Booking API validates slot. 4. Booking is stored in database. 5. Slot status changes to RESERVED. 6. User can pay for booking. 7. User can cancel booking. 8. Slot becomes AVAILABLE again.

## 7. Payment System Flow

1. User selects booking. 2. User selects payment method. 3. Backend generates transaction ID. 4. Payment entity is stored. 5. Payment status becomes PAID. 6. Admin can refund payment. 7. Payment status becomes REFUNDED.

## 8. Security System

- JWT Authentication - BCrypt password encryption - Role-based authorization - Protected admin routes - CORS configuration - Spring Security filter chain

## 9. API Testing

Swagger/OpenAPI is used for API testing. Main endpoints: - /api/auth/register - /api/auth/login - /api/bookings - /api/payments - /api/admin/\*

## 10. Database Structure

Main entities: - User - ParkingLocation - ParkingSlot - Booking - Payment Relationships: - One user can have many bookings. - One booking has one payment. - One location contains many parking slots.

## 11. Frontend Architecture

The frontend uses: - Pages - Components - Services - Context API - Protected routes - Reusable UI components

## 12. Future Improvements

- Real payment gateway integration - AI-based parking recommendations - Live slot monitoring - QR code parking access - Email notifications - Mobile application - Real-time WebSocket updates

## 13. Deployment

Frontend: - Vercel / Netlify Backend: - Railway / Render / AWS Database: - MySQL cloud hosting Environment variables should be used for: - JWT secret - Database credentials - API URLs